

Desafio 1: Mapa de características para o Perceptron

Ideia Central do código

Aprimorar o espaço para que um classificador puramente linear consiga resolver problemas não lineares.

Alterações no código

```
# 1. Calcula a norma quadrática
raio_quadrado = (Xs ** 2).sum(dim=1, keepdim=True)

parts = [Xs, raio_quadrado]
```

Soma dos quadrados substitui o individual. Resolve os círculos e esfera.

```
# 2. Termo de produto cruzado entre as duas primeiras variáveis de entrada
if self.use_cross:
    cross = (Xs[:, 0] * Xs[:, 1]).unsqueeze(1)
    parts.append(cross)
```

Adiciona o termo de produto cruzado a fronteira de decisão hiperbólica, vira uma decisão linear trivial baseada no sinal de um único peso.



```
# 3. Projeções de Fourier Multiescala (Random Fourier Features - RFF), blindar o determinismo)
Xs_64 = Xs.double()
W_64 = self.W.double()
b_64 = self.b.double()
```

Mudança do calculo das projeções para 64 bits temporariamente, antes de retornar para 32 bits.

```
rff_64 = self.rff_scale * torch.cos(Xs_64 @ W_64 + b_64)
rff = rff_64.float() # Retorna pro float32 esperado
```

Projeta os dados utilizando três larguras de banda distintas, preenche todo o orçamento restante com projeções de Fourier.

Score Final (S): **105.4** (Meta 100 superada. Entrou na faixa de bônus).

